

Unit 8: Generics

Generics

Generics means parameterized types.

The idea is to allow type (Integer, String, ... etc., and user-defined types) to be a parameter to methods, classes, and interfaces. Using Generics, it is possible to create classes that work with different data types."

Advantages of using Generics

1. **Code Reuse**
Write a class, method, or interface once and use it with different data types without rewriting the code.
2. **Type Safety**
Generics ensure that only the correct type is used, and any mismatch is caught at compile time—not at runtime. This helps prevent `ClassCastException`.
3. **Less Code**
Avoid writing duplicate code for each type. For example, one generic `Box<T>` can replace multiple versions like `IntBox`, `StringBox`, etc.
4. **Avoid Subclassing**
Without Generics, you'd have to create a new subclass for each data type. With Generics, one flexible class works with all types, reducing complexity.
5. **Stronger Typing**
The compiler knows exactly what type you're using, so it can provide better warnings, suggestions, and error checking.
6. **More Efficient Code**
Generics reduce the need for casting and manual type checks, improving both performance and readability.
7. **Easier Maintenance**
Generic code is more modular and less error-prone, making it easier to update and maintain.
8. **Improved Code Clarity**
Generics help express your intent more clearly. For example, `List<String>` immediately tells you the list is supposed to hold Strings only.

Generic Classes

A generic class in Java is a class that allows you to define the type of data it works with at compile time, using type parameters.

This means you can write one class that can be reused with different data types, without losing type safety.

Note: Generics work with reference types (e.g., Integer, String, List<T>), not primitive types like int or double. To use primitives, Java provides wrapper classes like Integer, Double, etc.

Example of Generic Class with Single Type Parameter:



```
public class Employee<T> {  
    private T employeeId;  
  
    public Employee(T employeeId) {  
        this.employeeId = employeeId;  
    }  
  
    public T getEmployeeId() {  
        return employeeId;  
    }  
  
    public void setEmployeeId(T employeeId) {  
        this.employeeId = employeeId;  
    }  
  
    @Override  
    public String toString() {  
        return "Employee{" + "employeeId=" + employeeId + '}';  
    }  
}
```

Example of Generic Class with Multiple Type Parameters:

```
public class Student<U, V> {  
    private U studentId;  
    private V studentName;  
  
    public Student(U studentId, V studentName) {  
        this.studentId = studentId;  
        this.studentName = studentName;  
    }  
  
    public U getStudentId() {  
        return studentId;  
    }  
  
    public V getStudentName() {  
        return studentName;  
    }  
  
    public void setStudentId(U studentId) {  
        this.studentId = studentId;  
    }  
  
    public void setStudentName(V studentName) {  
        this.studentName = studentName;  
    }  
  
    @Override  
    public String toString() {  
        return "Student{" + "studentId=" + studentId + ", studentName=" + studentName + '}';  
    }  
}
```

Example of Using Generic Class:



```
public class Main {  
    public static void main(String[] args) {  
        Employee<String> employee = new Employee<>("E123");  
        System.out.println(employee.getEmployeeId()); // prints "E123"  
  
        Student<Integer, String> student = new Student<>(1, "John Doe");  
        System.out.println(student.getStudentId()); // prints 1  
        System.out.println(student.getStudentName()); // prints "John Doe"  
    }  
}
```

Generic Method

A generic method is a method that defines its own type parameter(s), making it possible to work with different types while still maintaining type safety.

Unlike a generic class (where type parameters are declared at the class level), the type parameters in a generic method are declared just before the return type and are only accessible within that method.

Example of Generic Method



```
public <T> void printArray(T[] array) {  
    for (T element : array) {  
        System.out.println(element);  
    }  
}
```

Explanation

In this example, the method declares a type parameter `<T>` before the return type `void`, indicating it's a generic method. The parameter `T[]` array means the method accepts an array of any type `T`. This allows the method to work with arrays of different types like `String[]`, `Integer[]`, or any other object type. The compiler ensures type safety by making sure the array elements are all the type `T`.

Example of Using Generic Method:

<https://sawin.com.np>

<https://sawin.com.np>

<https://sawin.com.np>



```
public class Main {  
    public static <T> void printArray(T[] array) {  
        for (T element : array) {  
            System.out.println(element);  
        }  
    }  
  
    public static void main(String[] args) {  
        Integer[] intArray = {1, 2, 3};  
        String[] strArray = {"A", "B", "C"};  
  
        printArray(intArray); // Works with Integer  
        printArray(strArray); // Works with String  
    }  
}
```

Explanation

This code defines a generic method called `printArray` inside the `Main` class.

The method `<T> void printArray(T[] array)` declares a type parameter `<T>` before the return type (`void`), meaning it can work with any type `T`.

The parameter `T[] array` is an array of elements of type `T`.

Inside the method, a for-each loop goes through each element in the array and prints it.

Generic Constructor

A generic constructor is a constructor that defines its own type parameters, just like a generic method. This means the constructor can work with different types independently of whether the class itself is generic.

The type parameter declared in the constructor is local to that constructor only and doesn't affect the whole class.

Why use a generic constructor?

- It allows you to create objects with flexibility for different types right at construction time.
- You get type safety because the compiler checks the types during compilation.
- It increases code reuse by avoiding multiple constructors for different types.

Example



```
public class Example {  
    public <T> Example(T value) {  
        System.out.println("Value: " + value);  
    }  
}
```


Explanation

The class Example has a generic constructor — the constructor itself declares a type parameter `<T>`.

`<T>` means this constructor can accept a parameter value of any type.

When you create a new Example object, you pass in a value of any type (Integer, String, or even a custom class).

Inside the constructor, the passed value is printed out.

Important points:

The `<T>` before the constructor name declares that only this constructor is generic, not the whole class.

You don't need to make the entire class generic to use generic constructors.

The compiler ensures type safety — the value passed matches the type `T` during construction.

Example of Using Generic Constructor



```
Example e1 = new Example(123);           // T is Integer  
Example e2 = new Example("Hello");       // T is String
```

Generics: Class vs Constructor

Aspect	Generic Class	Generic Constructor
Type Parameter Scope	The type parameter is used throughout the entire class.	The type parameter is only valid within the constructor or method.
Class vs Method	The class is generic, so you specify the type when creating an instance of the class.	Only specific methods or constructors are generic. The class itself might not be.
Example Use Case	When you want the entire class to be flexible and work with multiple types.	When you only need a specific constructor or method to be flexible with types.

Real-World Usage Scenarios

Use Generic Classes when:

- You need consistent type safety across multiple operations
- Building data structures (List, Set, Map)
- Creating repositories, DAOs, or services that work with specific entity types

Use Generic Constructors when:

- You need flexible initialization from various source types
- Converting between different data formats during object creation
- Building utility classes that can be initialized from multiple types
- Creating builders or factories with type-safe construction

Polymorphism in Generic

Polymorphism means “many forms” — the ability of code to work with different types through a common interface or superclass.

When combined with generics, polymorphism lets you write classes, methods, or interfaces that operate on various types without losing type safety or having to write duplicate code.

How Polymorphism is Used in Generics

1. You define a generic type parameter (like `<T>`).
2. The code works with any type you provide, whether it's a built-in class (like `String` or `Integer`) or your own class.
3. At runtime, the objects passed can be of different types, but the compiler ensures they match the specified generic type.

Example: Polymorphism with Generics in Java

<https://sawin.com.np>

<https://sawin.com.np>



```
// Parent class representing a generic Animal  
class Animal {  
    // Method to be overridden by subclasses to define specific sounds  
    public void speak() {  
        System.out.println("Animal speaks");  
    }  
}
```



```
// Dog class inherits from Animal and provides its own implementation of speak()  
class Dog extends Animal {  
    @Override  
    public void speak() {  
        System.out.println("Dog barks");  
    }  
}
```



```
// Cat class inherits from Animal and provides its own implementation of speak()  
class Cat extends Animal {  
    @Override  
    public void speak() {  
        System.out.println("Cat meows");  
    }  
}
```



```
// Generic class that can hold any type of Animal (or subclass of Animal)
// Demonstrates use of Generics combined with polymorphism
class AnimalBox<T extends Animal> {
    private T animal; // Holds an instance of a specific type of Animal

    // Setter method to assign an animal to the box
    public void setAnimal(T animal) {
        this.animal = animal;
    }

    // Getter method to retrieve the animal from the box
    public T getAnimal() {
        return animal;
    }

    // Calls the speak method of the contained animal instance
    // This is a polymorphic call – the actual method executed depends on the runtime type of 'animal'
    public void makeSound() {
        animal.speak();
    }
}
```



// Main class to run the program

```
public class Main {  
    public static void main(String[] args) {  
        // Create an AnimalBox that specifically holds Dog objects  
        AnimalBox<Dog> dogBox = new AnimalBox<>();  
        dogBox.setAnimal(new Dog()); // Put a Dog into the box  
        dogBox.makeSound(); // Calls Dog's speak(): prints "Dog barks"  
  
        // Create an AnimalBox that specifically holds Cat objects  
        AnimalBox<Cat> catBox = new AnimalBox<>();  
        catBox.setAnimal(new Cat()); // Put a Cat into the box  
        catBox.makeSound(); // Calls Cat's speak(): prints "Cat meows"  
    }  
}
```

Explanation

- Animal is the base class with a speak() method.
- Dog and Cat are subclasses overriding speak() to provide specific behavior.
- AnimalBox is a generic class constrained to hold any Animal subclass.
- makeSound() uses polymorphism to call the appropriate speak() method based on the actual animal type stored.
- The main method demonstrates creating boxes for different animals and invoking their sounds.

Practical

1. Write a generic class Box that can hold any type of object. Demonstrate how generics can help prevent runtime type errors by trying to add incompatible types without using generics.
2. Write a generic method printArray that prints all elements of an array. Test this method with arrays of different types.
3. Write a generic method swap that swaps the positions of two elements in an array. Demonstrate this method with arrays of different types (e.g., Integer, String).
4. Create a generic class Pair that holds two values of potentially different types. Add methods to get the values and set new values. Demonstrate its usage with different type pairs.
5. Implement a generic Stack class with basic stack operations (push, pop, peek).
6. Write a generic class Container that holds an array of objects. Provide methods to add, get, and remove elements from the container. Ensure that the container dynamically resizes as needed.

Resources

<https://www.youtube.com/watch?v=K1iu1kXkVoA>

<https://www.javatpoint.com/generics-in-java>

<https://www.geeksforgeeks.org/generics-in-java/>

<https://www.javacodegeeks.com/2015/03/polymorphism-in-java-generics.html>